

SRMS Drona — The Project Explained

Everything about SRMS Drona in easy words. No jargon. If you can read this page, you understand the project.

For the technical version with code, see [`TECHNICAL.md`](#).

1. What is it? (30 seconds)

SRMS Drona is a website where the college's office staff — clerks, accountants, library staff, lab assistants — can do their training online.

Today that training is scattered across WhatsApp forwards, shared drives, and printed notes. Drona puts it in one place. Staff log in, watch short lessons, take a small quiz, and get a certificate. Their department head can see who has finished and who has not.

Think of it as **a training tracker that also teaches the training.**

Who uses it: about 14 people right now — one admin, six department heads, seven staff.

2. The technology, in plain words

We picked tools that are **boring, well-known and hard to break** — because this system has to keep working after we graduate.

What we used	What it does	Why
Python	The programming language we wrote it in	Very common, lots of help available
Django	The main toolkit that builds the website	Comes with security already built in
PostgreSQL	The database — stores all the data	Handles many users at once properly
HTML + CSS	How the pages look	Loads fast even on a slow phone
Argon2	Turns passwords into uncrackable codes	Even if someone steals the database, they get useless codes
Google Gemini	An AI that helps write quiz questions	Saves the admin hours of typing
ReportLab	Makes the PDF certificates	Standard, reliable
Render	The cloud service that hosts the website	Free tier, enough for this scale
GitHub Actions	Automatically tests our code every time we save	Broken code cannot reach the live site

One thing we deliberately did NOT use: React or Node. A lot of projects use them. We didn't, because our users are on phones with slow connections, and pages that come ready-made from the server load far faster. Simpler technology that actually works for the user beats impressive technology that doesn't.

3. The architecture, in plain words

Architecture just means: how the pieces are arranged.

Think of a restaurant kitchen. Orders come in at the front, get prepared in the middle, and the finished plate comes out. Our system works the same way, in **four steps**:

Step	Layer	What it is	In plain words
1	Users	Staff, HOD, admin	The people using it
2	The pages	38 server-rendered screens	What you actually see on screen
3	The brain	Seven modules	The rules — what you are allowed to see
4	The data	PostgreSQL database	Where the facts are kept

Step 1 — Users

Three kinds of person: **staff** (learn), **HOD** (heads a department, sees their own staff), **admin** (runs the whole platform).

Step 2 — The pages (what you see)

The computer asks the server for a page. The server builds the **whole finished page** and sends it over. There is no app to install and nothing that breaks if the internet hiccups. 38 pages, all in English and हिन्दी.

Step 3 — The brain (where the rules live)

Seven separate mini-programs, each in charge of one job — accounts, courses, quizzes, certificates, reports, reminders, and admin records. **Nobody can see someone else's data**, and that rule is checked here, on the server — not by hiding buttons.

Step 4 — The data (where facts live)

The database. Content is stacked neatly:

Category → Course → Module → Lesson

A **Category** is a subject area. A **Course** is like "Fire Safety". A **Module** is a chapter. A **Lesson** is one small piece — a video, a PDF, or some text.

This neat stacking is the important part. Because lessons are counted individually, we can work out exactly how far someone has got — rather than asking them to tick a box.

4. The core modules (the seven jobs)

Think of each app as a department with one responsibility.

Module	Its one job	In plain words
Users	Accounts and logins	Remembers who everyone is, and whether they're staff, a head, or an admin
Courses	Content, progress, video	Holds the lessons, and works out how far each person has got
Quizzes	Tests and AI help	Runs the questions, and helps the admin write new ones
Certificates	Proof of finishing	Makes the PDF certificate and the QR code
Analytics	Reports	Shows how far people got, and lets HR download the data
Notifications	Reminders	Nudges people who haven't finished mandatory training
Management	Record keeping	Logs who did what, for auditing

Why split into seven instead of one big program? Because when something breaks, we know exactly where to look — and the tests for certificates don't have to know anything about reports.

5. The core capabilities (what it can actually do)

For staff

- **Log in** with their **employee ID** — not an email address
- **See their dashboard** — which courses, how far done
- **Watch lessons** — and the video **remembers where you stopped**
- **Switch to हिन्दी** — the whole site, and the choice is remembered
- **Take a quiz** — pass mark is 70%
- **Get a certificate** — with a QR code, downloadable as PDF

For department heads

- Everything above, for themselves
- **See their own department's** progress
- **Add staff to courses** in bulk

For admins

- Everything above
- **Create and edit courses** — add lessons, videos, PDFs
- **Create accounts** and give people their role
- **See reports** across the whole college
- **Download reports** as CSV to put into a spreadsheet
- **Use AI to write quiz questions** from an SOP document

The two cleverest features

1. **The video remembers your place.** Close the tab, come back tomorrow, and it picks up at the same second. This works because the server understands partial requests — instead of re-downloading the whole video, your browser asks for just the part it needs. That's also why you can drag the timeline smoothly.
 2. **The QR code is real proof.** Every certificate carries a QR code. Anyone scanning it opens a public page that confirms the certificate is genuine — no login needed. But the page shows only the **first name and last initial** (Anant S.), so a certificate can be shown publicly without exposing the person's full name.
-

6. How someone actually uses it

The whole journey, in plain words:

1. **Admin creates an account** for a staff member, and enrolls them in a course.
 2. **Staff logs in** with employee ID and password.
 3. **Dashboard shows** their courses and how far done.
 4. **They open a course** and read the lessons. Videos remember their place.
 5. **The system counts** what they genuinely finished. It works this out on the server, so nobody can cheat by editing the page.
 6. **They take the quiz.** They need 70% to pass.
 7. **Certificate appears** — but only if they finished ****all the lessons *and* passed the quiz****. Both, not either.
 8. **They download the PDF.** It has a QR code.
 9. **HR checks the report** to see who is done.
-

7. Three things that make it trustworthy

Progress is worked out by the server, not claimed by the user

If you change something in your browser, the server ignores it. You cannot mark yourself 100% done — the server counts the lessons you actually completed.

Watching is measured, not assumed

You could drag a video to the end in one click. So the system counts the time it has actually ***sent*** you, and ignores the "finished" flag your browser sends for video lessons. One heartbeat can only ever add 30 seconds, and the total can never go past the video's real length.

The tests never sleep

203 automated tests run **every single time** anyone saves code. If something breaks, the live website cannot be updated. That is why we can say the system works.

8. Three real bugs we found and fixed

We didn't just take a checklist and tick it off. We tried to **break our own system**, and found three genuine problems:

#	What was wrong	How we found it	Fix
1	Someone could upload a file that ran code in other people's browsers	Uploading a file that lied about its type	Now the filename and the file type must both agree, and the server decides the real type
2	Someone could try millions of passwords without getting blocked	The system trusted a header the user could fake	Now it uses the real connection address it cannot be tricked into
3	Real login details were published in our public code repository	Reading our own README	All 14 accounts got new unique passwords, and the details moved to a secure password manager

The pattern: all three were found by **testing a control against a real attacker**, not by reading that the control existed. And each one now has an automated test, so it cannot come back.

9. What is honest about the limits

We think a project that admits its gaps is stronger than one that claims perfection. So:

- **Emails are not being sent yet.** The reminder feature is fully built, but the college has not given us email server access, so no reminders go out. The on-screen reminders are also switched off by default until that is fixed.
 - **We have not tested it under heavy load.** We know it works with the people using it now. We do not claim it is proven for hundreds at once.
 - **Videos are stored on the server's own disk.** Fine now, but before real scale it should move to proper cloud storage.
 - **The screen reader / accessibility pass is not finished.**
-

10. The short version

SRMS Drona is a training website for college office staff, built with Django and PostgreSQL, hosted free in the cloud.

It is organised in four layers — users, pages, rules, database — split into seven focused modules so problems are easy to find and test.

It is bilingual, remembers where you stopped watching, and issues QR-verifiable certificates only after you finish the lessons and pass the quiz.

Its progress is calculated on the server, so it cannot be faked. It runs 203 automatic tests, has no known security problems, and is live at dronav2.onrender.com.

Where to look next

If you want...	Open
The simple overview	This page
Every feature explained	`PROJECT-GUIDE.md`
How the code works, in detail	`TECHNICAL.md`
What to say in a presentation	`presentation/SCRIPT.md`
The full written report	`report/report3.0.pdf`
The actual code	< https://github.com/dgexplores/DRONA >
The live website	< https://dronav2.onrender.com >